

Lecture 1 (INT 332) BASICS OF DEVOPS

Revolutionizing Containerization

Why Devops



Understanding the concepts of Virtualization and Containerization

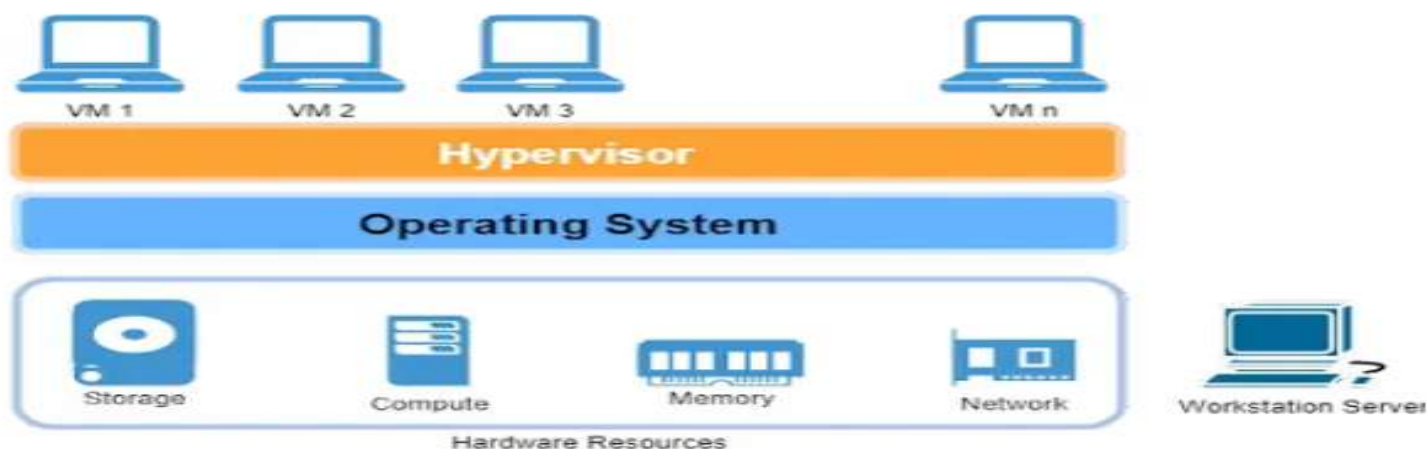


Imagine a peaceful independent house.

The Apartment Era (Virtualization)



Infrastructure followed the same path.



One physical server was divided into multiple **virtual machines**.
Each VM felt like its own house — with its own OS, memory, and CPU.

Utilization improved.
Costs reduced.

Virtualization



- ◎ With the changing times, businesses started looking for solutions to reduce overhead costs, enhance scalability, and standardize application deployment process.

- ◎ They started considering the following two approaches to reduce costs –
 1. Virtualization
 2. Containerization

What is Virtualization?



- ◎ Virtualization is the process of **partitioning** a physical server into multiple virtual servers.
- ◎ The process of partitioning is carried out using a software called **'hypervisor'**.
- ◎ After partitioning, the virtual servers act and perform just as a physical server.
- ◎ Essentially, it means **using the same hardware** set-up more efficiently, thereby freeing the resources for other tasks or retiring the resources altogether.

Advantages of Virtualization

- ⦿ **Enhanced performance**
- ⦿ **Promotes agile IT infrastructure**
- ⦿ **Promotes use of resources in optimum manner**
- ⦿ **Better disaster management if any physical hazards take place to resources**
- ⦿ **Better security as the infected VM can be isolated from other VMs and the host server**
- ⦿ **Space saving**
- ⦿ **The capital investment cost on hardware is saved, and thereby maintenance cost is saved, hence overall cost savings for a business.**

Drawbacks of Virtualization which lead to concept of containerization



- 1. Heavy Resource Overhead**
- 2. Slow Boot Time and Poor Agility**
- 3. Inefficient Scalability**
- 4. Limited Application Portability**
- 5. Higher Management and Maintenance Complexity**
- 6. Increased Infrastructure Cost**

These drawbacks directly led to the evolution of containerization, which offers lightweight isolation, faster deployment, better portability, and seamless integration with DevOps and microservices architectures.

PG Living (Containerization)



High rent.
Fast life.
Limited space.

People started choosing **PGs and hostels.**

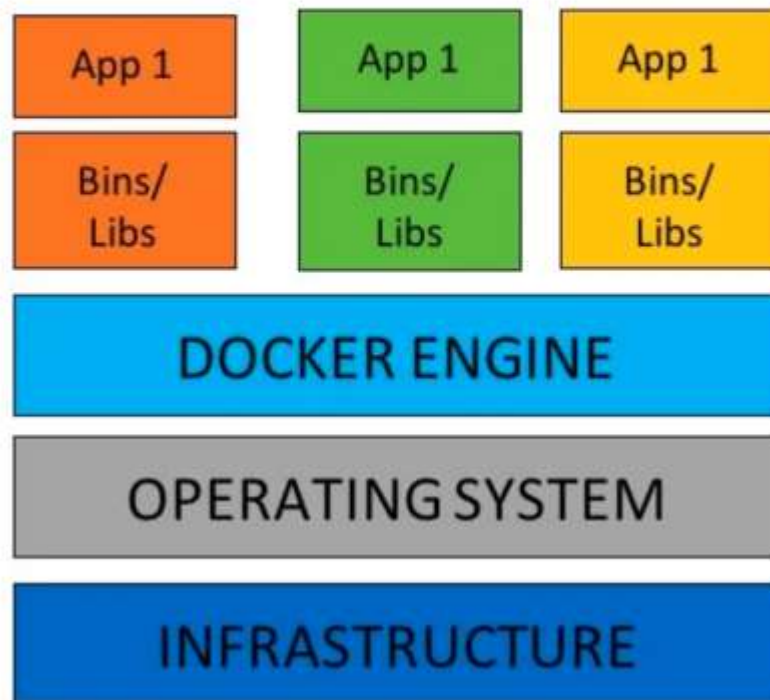
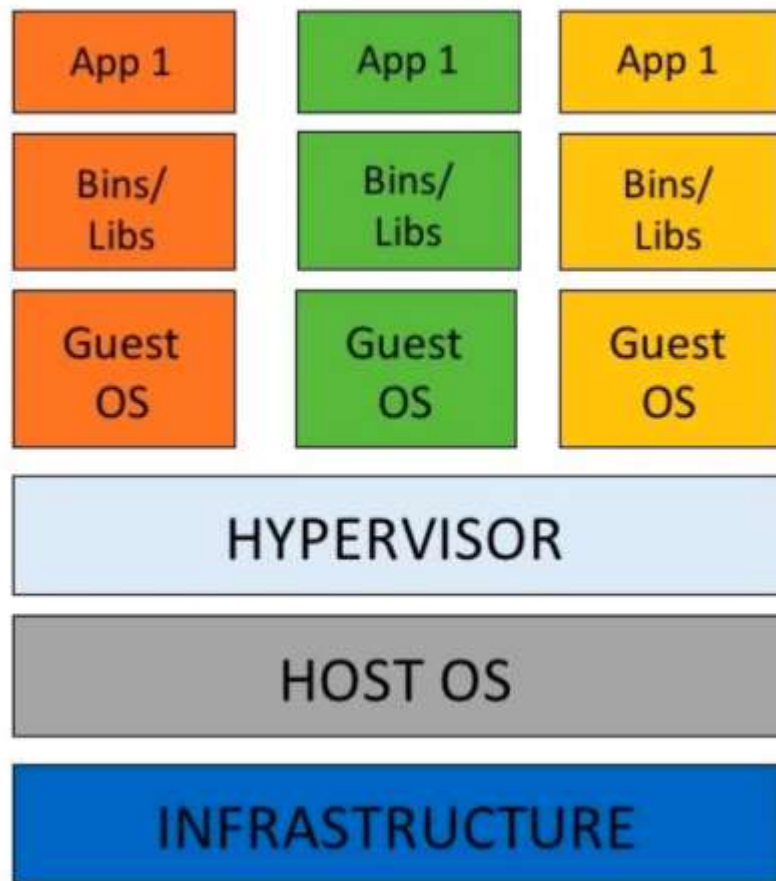
You don't own the building.
You don't manage electricity.
You don't worry about maintenance.

That's exactly what containers do.



- Containers don't carry an entire operating system. They use bare minimum OS.
- They share the host OS and only bring what the application needs.
- They start in seconds. They consume minimal resources.
- You can run dozens — even hundreds — on the same machine.
- Earlier the applications were monolithic , so physical servers were apt,
- later there were components split like backend, frontend, db ,so VM's took care of hosting.
- Now all the applications are turning to microservices, so the containerization tools like docker will serve the purpose.
- Also the portability problem is also solved using the containerization

Need of containers



Container Runtime

- A container runtime is the software responsible for creating, starting, stopping, and managing containers on a host system.
- It acts as the execution engine that turns a container image into a running container.

In simple words:

“Container runtime is what actually runs containers.”

Need for Container Runtime

- A container image is just a read-only package (application + dependencies).

To run it, the system needs a runtime to:

- Create an isolated environment
- Apply resource limits
- Start the application process
- Monitor and stop it when required

Types of Container Runtimes

1. High-Level Runtimes

Used by users and DevOps tools.

Docker Engine

containerd

CRI-O (Kubernetes-focused)

These handle:

- **Image pulling**
- **Networking**
- **Storage**
- **Container lifecycle**

2. Low-Level Runtimes

- Responsible for actually running the container process.
- runc (OCI-compliant)

These directly interact with:

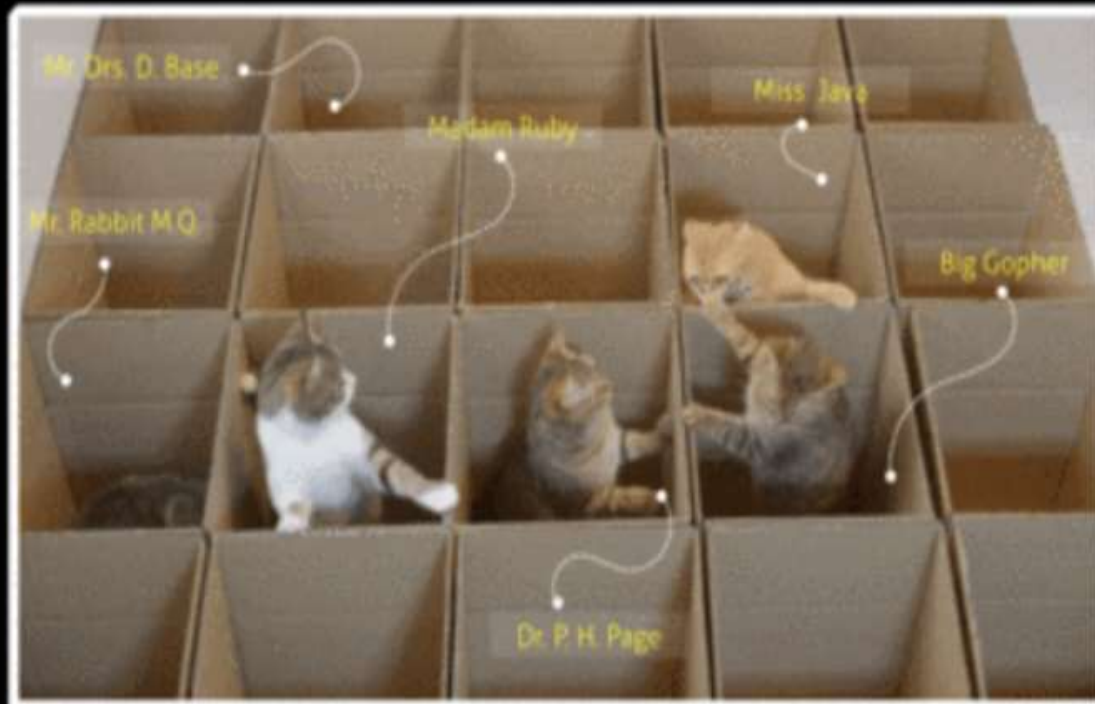
- Linux kernel
- Namespaces
- cgroups

Real-World Analogy

- **Container Image → Packed lunch box**
- **Container Runtime → Person who opens it, serves food, and cleans up**

Process Isolation & Namespaces

- **Process isolation ensures that each container runs as if it is alone on the system, even though multiple containers share the same OS kernel.**
- **This isolation is achieved using Linux namespaces.**



containers isolate a process

Interpretation of Picture

- As shown is this little cat picture, containers are isolated but using networking they still **discover** each other and consume the services if needed.

Namespaces



- Namespaces are Linux kernel features that partition system resources so that processes see only what belongs to them.
- *“Namespaces create the illusion of a separate system for each container.”*

Namespaces Used in Containers

1. PID Namespace (Process Isolation)

- Each container has its own process tree
- Process IDs inside container start from 1
- A container cannot see host or other container processes

E.g: Inside a container, the main app runs as PID 1, just like init/systemd in a VM.

2. Network Namespace



- **Each container gets:**
 - Its own IP address
 - Its own ports
- **Containers can run apps on the same port without conflict**
- **Example:**
Multiple Nginx containers all listen on port 80 internally.

3. Mount Namespace

- Each container has its own filesystem view
- File changes inside container do not affect host

4. UTS Namespace

- Allows each container to have its own hostname

5. User Namespace (Advanced)

- Maps container users to non-root users on host
- Improves security

Namespaces are like separate cabins in a train:

- **Same engine (kernel)**
- **Different passengers**
- **No visibility between cabins**

Control Groups (cgroups) for Resource Limits

Control Groups (cgroups) are Linux kernel features that limit, control, and monitor resource usage of processes.

They ensure that:

- No container can monopolize system resources
- Fair sharing of CPU, memory, and I/O

“Namespaces isolate, cgroups control.”

Need of cgroups

Without cgroups:

- One container could consume all CPU or RAM
- Host and other containers could crash

Types of Resources Controlled by cgroups

1. CPU

- Limit CPU usage
- Assign CPU shares

2. Memory

- Set maximum memory usage
- Kill container if limit exceeded

3. Disk I/O

- Limit read/write speeds

4. Network (Indirectly)

- Through traffic shaping mechanisms

cggroups are like electricity meters in apartments:

- **Each flat has a limit**
- **One tenant cannot use all power**

Finally for namespaces and cgroups

Containers are lightweight because they do not virtualize hardware.

Instead, a container runtime uses Linux namespaces for process isolation and cgroups for resource control, allowing multiple isolated applications to run efficiently on a shared operating system kernel.



Without cgroups (Problem Scenario)

HOST SYSTEM

CPU: 100%

Memory: 16 GB

Container A (No Limits)

Uses **90%** CPU

Uses **14 GB** RAM

Container B
CRASHED

Container C
CRASHED

With cgroups (Controlled)

HOST SYSTEM

CPU: 100%

Memory: 16 GB

Container A (cgroups)

CPU Limit : 30%

RAM Limit: 4 GB

Container B

CPU: 30%

RAM: 6 GB

Container C

CPU: 40%

RAM: 6 GB

Kernel Enforces Limits



cgroups
Resource Limits



Namespaces
Isolation

No Container Can Monopolize Resources

Today's Task

Task 1: A company uses containers for microservices. One faulty container crashes the host due to high memory usage.

Questions:

- Which container feature was misconfigured or missing?
- Which kernel mechanism should have prevented this?
- Would namespaces alone solve this issue? Why?

Task 2: A server is running three containers:

- **Container A runs a CPU-intensive task**
- **Container B runs a web server**
- **Container C runs a database**

Questions:

- **Which Linux feature prevents Container A from consuming all CPU?**
- **Which feature ensures Container B cannot see processes of Container C?**
- **Which namespace allows all three containers to use port 80 internally?**

Container Image

A container image is:

- A read-only blueprint
- Contains:
 - Base OS (minimal)
 - Application code
 - Libraries & dependencies
 - Runtime
 - Configuration

Images are immutable – once built, they do not change.

Example

python:3.11-slim is the image

This image already includes:

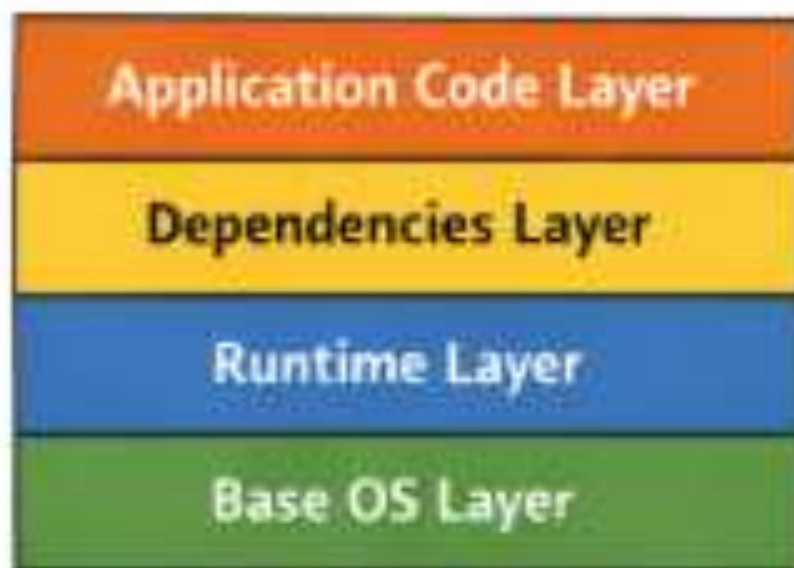
- **Linux OS**
- **Python 3.11**
- **Standard Python libraries**

Each image is built as a stack of layers.

Characteristics of Layers

- **Each instruction in a Dockerfile = one layer**
- **Layers are:**
 - **Immutable**
 - **Cached**
 - **Reused across images**

Image Layers



Faster Builds

- Docker reuses unchanged layers (cache)

Smaller Storage

- Same layers shared between images

Faster Deployment

- Only missing layers are downloaded

Container Image vs Container

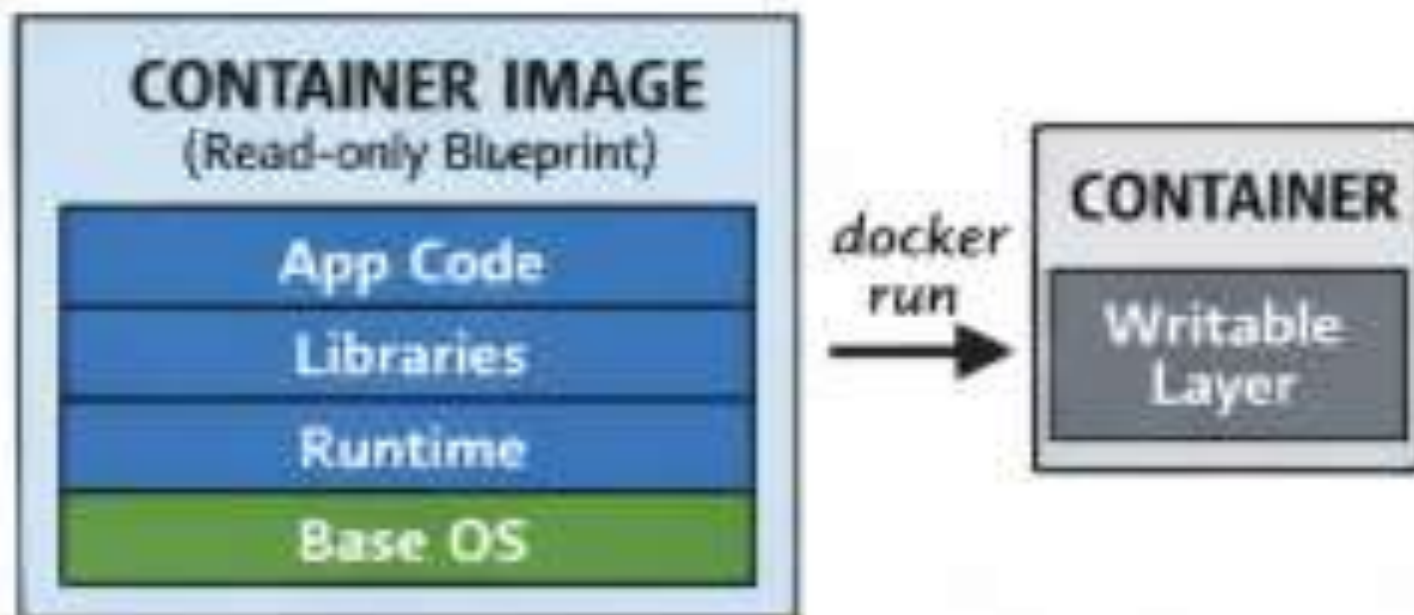


Image Registry



An image registry is a centralized repository used to store, manage, version, and distribute container images.

It enables:

- **Sharing images across teams and environments**
- **Consistent deployment in development, testing, and production**
- **Integration with CI/CD pipelines**

In DevOps, an image registry plays the same role as:

- **GitHub → for source code**
- **Maven Central → for Java libraries**

Why Image Registries are Required



Without a registry:

- Each system must build images independently
- High inconsistency across environments
- Manual software installation
- No version control of application environments

With a registry:

- Build once, deploy everywhere
- Faster deployments
- Version-controlled environments
- Rollback capability
-

3.1 Public Image Registries

Public registries allow open access to images.

Characteristics:

- **Images are publicly visible**
- **Anyone can pull images**
- **Often free**
- **Used for base images and open-source applications**

Private Image Registries



Private registries restrict access using authentication and authorization.

Characteristics:

- Secure and controlled access
- Used in enterprises
- Stores proprietary applications
- Integrated with IAM and RBAC

Examples:

- AWS Elastic Container Registry (ECR)
- Azure Container Registry (ACR)
- GitHub Container Registry (GHCR)
- On-premise private registry

Image Naming Convention

Images follow a standard naming

format:<registry>/<namespace>/<image>:<tag>

Example:docker.io/harpreet/webapp:v1

Component	Meaning
registry	Location of image
namespace	User or organization
image	Application name
tag	Version or label

Tags and Versioning

Tags identify different versions of the same image.

Common Tagging Strategies

Tag	Purpose
latest	Default (not recommended in production)
v1, v2	Versioned releases
dev	Development build
qa	Testing build
prod	Stable production build

Image Distribution Process

Image distribution refers to how images move from developers to production systems.

Distribution Flow:

- Developer builds an image locally
- Image is pushed to a registry
- Registry stores the image securely
- Servers pull the image when required
- Containers are created from the image

Push and Pull Operations

Push Operation

- Uploads image layers from local system to registry
- Requires authentication
- Only new layers are pushed

Pull Operation

- Downloads image layers from registry
- Uses caching to avoid re-downloading existing layers
- Faster on repeated deployments

Role of Image Registries in CI/CD Pipelines

In modern DevOps pipelines:

- **Code is committed to Git**
- **CI system builds the container image**
- **Image is pushed to registry**
- **CD system pulls the image**
- **Application is deployed automatically**

In Actual, Registry acts as the bridge between CI and CD.

Comparison Between Public and Private Registry

Feature	Public Registry	Private Registry
Access	Open	Restricted
Security	Lower	High
Cost	Mostly free	Paid
Use Case	Learning, open source	Enterprise, production

QUIZ



1.What is the primary purpose of an image registry?

- A. To execute containers**
- B. To store and distribute container images**
- C. To build container images**
- D. To monitor container performance**

2. Which component of an image name represents the version of the image?

- A. Registry**
- B. Namespace**
- C. Image name**
- D. Tag**

3. Which type of image registry is typically used in enterprise environments?

- A. Public registry**
- B. Open registry**
- C. Private registry**
- D. Anonymous registry**

4. In CI/CD pipelines, the image registry mainly acts as:

- A. Source code repository**
- B. Testing environment**
- C. Bridge between CI and CD**
- D. Container runtime**

Introduction to Docker Hub

Centralized repository for Docker images

Features:

- **Public and private repositories**
- **Official images**
- **Automated builds**
- **Importance in DevOps**

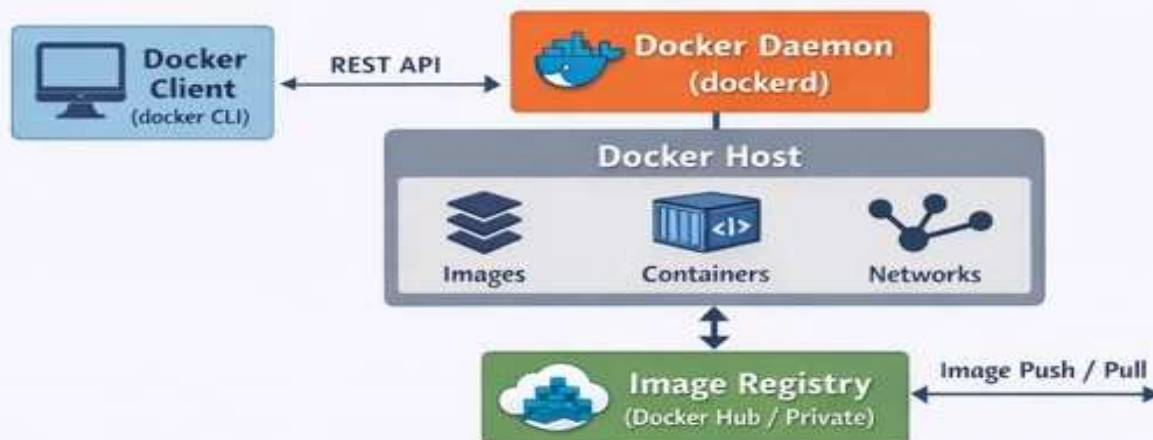
Docker Architecture

Key Components:

- - Docker Client
- - Docker Daemon
- - Docker Images
- - Containers
- - Docker Registry (Docker Hub)

Docker Architecture

Docker Architecture Overview



Analogy:



Example Workflow:



Docker Life Cycle

Stages:

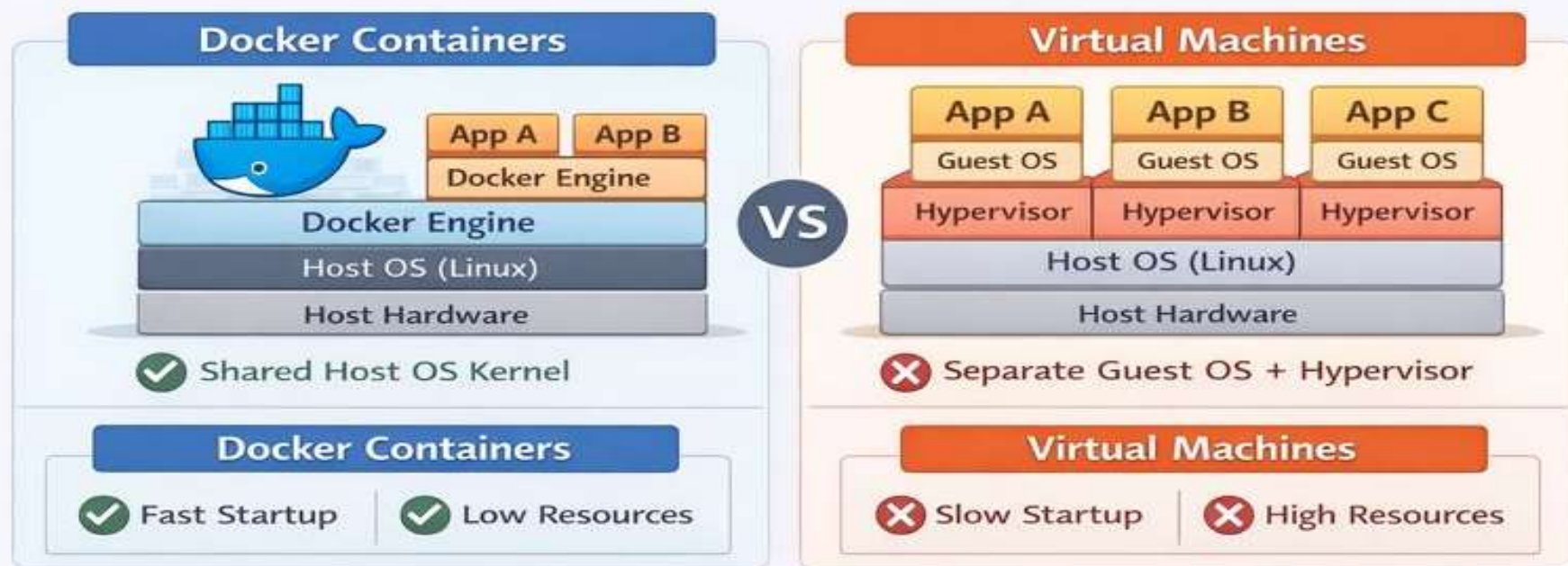
- - Build: Create an image
- - Ship: Share images via Docker Hub
- - Run: Start containers

Container states:

- - Created
- - Running
- - Paused
- - Stopped

Why is Docker faster than VM?

- Containers share the Host OS Kernel
- No Guest OS boot needed
- Less resource usage & faster startup



Conclusion



Key takeaways:

- Docker simplifies software deployment
- Docker Hub enhances collaboration
- Docker's architecture and lifecycle enable efficient workflows
- Hyper-V integration broadens Docker's capabilities



Basic Docker commands

Course Code: INT332

1

Finding the version



One of the first things you want to know is how to find the installed docker version.

- `$ docker --version`
- Docker version 18.09.6, build 481bc77

2. Docker info



- Get detailed information about docker installed on the system including the kernel version, number of containers and images, etc.
- **\$ docker info**
 - Containers: 3
 - Running: 1
 - Paused: 0
 - Stopped: 2
 - Images: 3
 - Server Version: 18.09.6
 - Storage Driver: overlay2
 - Backing Filesystem: extfs
 - Supports d_type: true Native Overlay Diff: true Logging Driver:

3. Checking History



- Shows the history of a docker image with the image name mentioned in the command

\$ docker history httpd

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
--------------	----------------	-------------------	-------------	----------------

4 . Downloading Image



When we need to pull the docker image from dockerhub (docker repository) then ,the following example of pulling the the image and here we are taking Apache HTTP server image.

\$ docker pull httpd

```
Using default tag: latest

latest: Pulling from library/httpd

f5d23c7fed46: Pull complete

b083c5fd185b: Pull complete

bf5100a89e78: Pull complete

98f47fcaa52f: Pull complete

622a9dd8cfed: Pull complete

Digest: sha256:8bd76c050761610773b484e411612a31f299dbf7273763103edbdba82acd73642

Status: Downloaded newer image for httpd:latest
```

5. Images



To list all the docker images pulled on the system with image details such as TAG/IMAGE ID/SIZE etc.

\$ docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
httpd	latest	ee39f68eb241	2 days ago	154MB
hello-world	latest	fce289e99eb9	6 months ago	1.84kB
sequenceiq/hadoop-docker	2.7.0	789fa0a3b911	4 years ago	1.76GB

- The ‘**docker run**’ command is used to create and start a new container from a **Docker** image. The basic syntax of the ‘**docker run**’ command is as follows:
- **docker run [OPTIONS] IMAGE [COMMAND] [ARG...]**
- *where,*
- **OPTIONS:** *optional flags that configure the behaviour of the container, such as setting environment variables, mounting volumes, etc.*
- **IMAGE:** *the name of the Docker image to be run.*
- **COMMAND:** *the command to be executed when the container is started. If not provided, the default command specified in the Docker image will be used.*
- **ARG:** *optional arguments passed to the command.*

-d means

Options Available for executing Docker Run Command



- Some commonly used options for executing the docker run command inside a container:
- *-it: Runs the container in interactive mode, allowing you to interact with the container through the command line.*
- *-d: Runs the container in the background.*
- *-- name: Specifies a name for the container.*
 - rm: Automatically removes the container when it exits.*
- *-p: Maps a host port to a container port.*
- *-e / --env: Sets an environment variable inside the container.*
- *-v: Mounts a host volume inside the container.*

1st Example : Running a Simple Image:



docker run httpd echo “Hello, World!”

- This command runs a new container from the ‘**httpd**’ image and executes the ‘**echo**’ command with the argument “Hello, World!”.

Running a Container with a Specific Name:

- ***\$ docker run --name my-container httpd echo “Hello, World!”***
- ***--name** : this option is to give the name of container*
- ***my-container**: it is the name of container*
- ***Echo**: it is the command*
- ***“hello,world!”** – it is argument used*
- This command runs a new container with the specified name “my-container” from the **httpd** image and executes the echo command with the argument “Hello, World!”.

What are Environment Variables



Environment variables are **key–value pairs** used by applications to:

- Read configuration
- Store secrets (usernames, passwords)
- Control application behavior **without changing the image**
- *Environment variables allow us to configure containers dynamically.*

Need of environment variables in Docker



Without -e:

- Configuration must be **hard-coded** in the image
- Need to rebuild image for every change

With -e:

- Same image → different behavior
- Secure & flexible configuration
- Widely used in **microservices & cloud deployments**

3rd Example : Setting Environment:



- **docker run -e MY_VAR=value httpd env**
- This command sets an environment variable '**MY_VAR**' to the value "**value**" and runs a new container from the '**httpd**' image.

The '**env**' command is executed to display the list of environment variables in the container.

2nd Example: Run a container from Image and display the variable name inside it.

`docker run -it -e MY_NAME=Harpreet ubuntu bash`

It will open the container and you can check the env variable inside it as:

Echo \$ MY_NAME

Kindly note :

MY_NAME is created inside the container

Host system is not affected

Multiple Environment Variables



- `docker run -e APP_ENV=production -e APP_VERSION=1.0 nginx`

Same for MySQL Container

```
docker run -d \  
-e MYSQL_ROOT_PASSWORD=root123 \  
-e MYSQL_DATABASE=college \  
-e MYSQL_USER=admin \  
-e MYSQL_PASSWORD=admin123 \  
mysql:8
```

Without these variables, MySQL container will fail to start

Next is : Passing Environment Variable from Host System



Ex#1:

Step 1: Set variable on host

```
export APP_PORT=8080 (Ubuntu)
```

```
Powershell - $env:APP_PORT=8080
```

```
Check host variable - echo $env:APP_PORT
```

Step 2:

```
docker run -e APP_PORT=$env:APP_PORT nginx env
```

```
docker run -e APP_PORT=$APP_PORT nginx env
```



Another way : Using --env-file (Clean & Professional Way)

Create .env file

DB_HOST=localhost

DB_USER=root

DB_PASS=secret

Then Run container

`docker run --env-file .env myapp`

4th Example



```
$ docker run -it -d httpd
```

Here :

- **-it** flag enables interactive mode, so you can interact with the container's command line
- The **-d** flag runs the container in the background and the **httpd** argument specifies the image name to use for the container. This command will pull the **httpd** image from Docker Hub if it is not already present on your local machine, create a container based on that image, and start it in the background.

Passing **Environment Variables** to Containers



Dynamic Configuration at Runtime

-p optional flag in run command



- **-p stands for *publish port*.**

It maps a **port on your host machine** to a **port inside the container**.

In simple words:

- **-p allows users outside the container to access an application running inside the container.**

By default:

- Containers run in an **isolated network**
- Applications inside containers are **NOT accessible from the host/browser**
- So if a web server is running **inside a container**, you **must expose it** using -p.

Syntax of -p



`docker run -p <HOST_PORT>:<CONTAINER_PORT> IMAGE`

Part	Description
<code>HOST_PORT</code>	Port on your system (Laptop/Server)
<code>CONTAINER_PORT</code>	Port where app runs inside container

EX#1:



- Without `-p`
- `docker run nginx`
- Try opening browser: But it is Not accessible
As no port mapping done here

Now try to Run Nginx with -p

- `docker run -p 8080:80 nginx`

What happens internally?

Browser → localhost:8080 → Docker Host → Container:80 → Nginx

- `docker run -d -p 3307:3306 --name mysql-test -e MYSQL_ROOT_PASSWORD=root123 mysql`

`docker exec -it mysql-test bash`

`mysql -h 127.0.0.1 -P 3306 -u root -p`

7 What's running?



- To lists all the docker containers are running with container details.
-
- **\$ docker ps**

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
09ca6feb6efc	httpd	"httpd-foreground"	36 seconds ago	Up 33 seconds	80/tcp	suspicious_b

TASK #1:



- Run Ubuntu container
- Pass `-e COLLEGE=CSE`
- Show `echo $COLLEGE`
- Stop container then check what happened

8. ps -a



To list all the docker containers running/exited/stopped with container details

- **\$ docker ps -a**

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
09ca6feb6efc	httpd	"httpd-foreground"	51 seconds ago	Up 49 seconds	80/tcp
2f6fb3381078	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -d"	2 weeks ago	Exited (137) 9 days ago	
9f397feb3a46	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -..."	2 weeks ago	Exited (255) 2 weeks ago	2122/tcp
9b6343d3b5a0	hello-world	"/hello"	2 weeks ago	Exited (0) 2 weeks ago	

9. exec



- Access the docker container and run commands inside the container in bash

\$ docker exec -it 09ca6feb6efc bash/env

09ca6feb6efc --- it is Id of the container

```
root@09ca6feb6efc:/usr/local/apache2# ls

bin  build  cgi-bin  conf  error  htdocs  icons  include  logs                modules

root@09ca6feb6efc:/usr/local/apache2#
```

Type *exit* and press enter to come out of the container.

10. Removing container



- Remove the docker container with container id mentioned in the command.

```
$ docker rm 9b6343d3b5a0
```

Run the below command to check if the container got removed or not.

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
09ca6feb6efc	httpd	"httpd-foreground"	About a minute ago	Up About a minute	80/tcp
2f6fb3381078	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -d"	2 weeks ago	Exited (137) 9 days ago	
9f397feb3a46	sequenceiq/hadoop-docker:2.7.0	"/etc/bootstrap.sh -..."	2 weeks ago	Exited (255) 2 weeks ago	2122/tcp

11. Removing image



- Remove the docker image with the docker image id mentioned in the command

\$ docker rmi fce289e99eb9

```
Untagged: hello-world:latest
```

```
Untagged: hello-world@sha256:41a65640635299bab090f783209c1e3a3f11934cf7756b09cb2f1e02147c6ed8
```

```
Deleted: sha256:fce289e99eb9bca977dae136fbe2a82b6b7d4c372474c9235adc1741675f587e
```

```
Deleted: sha256:af0b15c8625bb1938f1d7b17081031f649fd14e6b233688eea3c5483994a66a3
```

```
geekflare@geekflare:/home/geekflare$
```

- To remove the image forcefully

```
$ docker rmi -f <image id>
```

```
docker rmi -f 92fa43a2ff60503fdf250
```

```
Untagged: httpd:latest
```

```
Untagged:
```

```
httpd@sha256:7765977cf2063fec486b63ddea574faf8fbed285f  
2b17020fa7ef70a4926cdec
```

```
Deleted:
```

```
sha256:92fa43a2ff60503fdf250ef0a16d4170aa2a47c4b5a60d4  
27724283543a8792a
```


Practice Questions on Basic Docker Commands



- Run a Docker container named “DB-app” based on the “mongodb” image, and expose port 80 on the host to port 8082 on the container?

`docker pull mongo`

`docker run -d --name DB-app -p 80:8082 mongo`

- You are a DevOps engineer in a startup company. The development team asks you to quickly deploy a simple web page that displays the message:

Use the official httpd Docker image.

Run the container so it is accessible on port **8080** of the host machine.

Create or update an HTML file inside the container to display the required message.

Verify the output using a command-line tool.

Properly stop and remove the container after testing.

- `docker pull httpd`
- `docker run -d --name my-apache -p 8080:80 httpd`
- `docker exec -it my-apache bash -c "echo '<h1>Hello from Docker Apache!</h1>' > /usr/local/apache2/htdocs/index.html"`
- `curl http://localhost:8080`
- `docker stop my-apache`
- `docker rm my-apache`

3. How would you use the docker run command with -it, -e, -v, and --name to:

- Set an environment variable APP_ENV=production.
- Bind a local directory /app/data to /data inside the container.
- Name the container my_app.



- `docker pull ubuntu`
- `mkdir C:\app\data`
- `docker run -it --name my_app -e APP_ENV=production -v C:\app\data:/data ubuntu /bin/bash`
- `echo $APP_ENV`
- `echo "Hello this is my first program" > /data/test.txt`
- `cat /data/test.txt`
- `echo "Second line added" >> /data/test.txt`
- `cat /data/test.txt`
- `exit`

Docker container commands:



1. Container Lifecycle Management

- **Run a container** (creates and starts it)
 - `docker run -d -p 80:80 --name my_container nginx`
- **Start a stopped container**
 - `docker start <container_id/name>`
- **Stop a running container**
 - `docker stop <container_id/name>`
- **Restart a container**
 - `docker restart <container_id/name>`
- **Pause a container**
 - `docker pause <container_id/name>`

...contd....

- **Unpause a paused container**
- `docker unpause <container_id/name>`
- **Kill a container (force stop)**
- `docker kill <container_id/name>`
- **Remove a container**
- `docker rm <container_id/name>`
- **Remove all stopped containers**
- `docker container prune`

2. Listing and Inspecting Containers



- **List all running containers**
- `docker ps`
- **List all containers (including stopped ones)**
- `docker ps -a`
- **Show detailed info about a container**
- `docker inspect <container_id/name>`
- **Display resource usage stats of a container**
- `docker stats <container_id/name>`
- **View container logs**
- `docker logs <container_id/name>`
- **Follow logs in real-time**
- `docker logs -f <container_id/name>`

3. Container Interaction with Host



- **Execute a command inside a running container**
- `docker exec -it <container_id/name> <command>`
- **Attach to a running container's interactive shell**
- `docker attach <container_id/name>`
- **Copy files between container and host**
 - From container to host:
 - `docker cp <container_id>:<container_path> <host_path>`
 - From host to container:
 - `docker cp <host_path> <container_id>:<container_path>`
- E.g:

Commands for CI with Host



- `docker exec -it Container ID ls /`
- `docker exec -it Container ID /bin/bash`
- `mkdir /data`
- `echo "Hello Docker" > /data/test.txt`
- `exit`
- `docker attach Container ID`
- `docker cp Container ID:/data/test.txt C:\Users\HP\Desktop\`
- `docker cp C:\Users\HP\Desktop\test.txt Container ID`
`:/data/sample.txt`
- `docker exec -it Container ID ls /data`
- `docker exec -it Container ID cat /data/sample.txt`

Task



- Inside this same running container, you are required to:
- Create a new directory named /project.
- Inside /project, create a file called report.txt.
- Add the following text inside the file:
This is our container interaction with our host machine.
- Verify the file content.
- Copy report.txt from the container to your Desktop.
- Create another file named notes.txt in the same /project directory.
- Append the text:
- Verify that both files exist inside /project.



- `docker exec -it
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224 mkdir /project`
- `docker exec -it
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224 sh -c "echo 'Docke Lab Assignment' >
/project/report.txt"`
- `docker exec -it
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224 sh -c "echo 'Container File Creation Test' >>
/project/report.txt"`
- `docker exec -it
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224 cat /project/report.txt`



- `docker cp`
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224:/project/report.txt C:\Users\HP\Desktop\
- `docker exec -it`
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224 sh -c "echo 'This is an additional file inside the
same container.' > /project/notes.txt"
- `docker exec -it`
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224 ls /project
- `docker exec -it`
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99
187beaf3224 cat /project/notes.txt

Practice questions:



- 1. You need to start a new container using the nginx image while setting an environment variable `ENV_MODE=production`. Write the docker run command to achieve this.
- 2. You have a running container named `my_app`, but it's not behaving as expected. You want to check its logs to debug any errors. Write the command to view its logs and follow new log entries in real-time.
- 3. A container named `web_server` was stopped. Write the command to:
a) Start the container again.
b) Stop the container when needed.

Basic commands used to create,inspect,verify , delete/remove the volumes

1. `docker volume create <volume name>`
2. `docker volume ls`
3. `docker volume inspect mydata`
4. `docker run -d -v <volumename>:<path where to store in image> --name <containername> <imagename>`

- `docker volume create mydata`
- `docker volume ls`
- `docker volume inspect mydata`
- `docker run -dit --name mycontainer -v mydata:/app/data ubuntu`
- `docker exec -it mycontainer ls /app/data`
- `docker inspect mycontainer`
- `docker rm -f mycontainer`
- `docker volume rm mydata`
- `docker volume prune`



- Create a Docker volume named projectdata.
- Verify that the volume has been created successfully.
- Inspect the volume and identify: The mount point (where Docker stores it on the host system).
- Run an Ubuntu container in detached mode:
Name the container project_container.
- Attach the volume projectdata inside the container at /app/data.
- Inside the running container: Create a file named report.txt inside /app/data.
- Add the text: This is my first volume.
- Stop and remove the container.
- Run a new Ubuntu container: Name it project_container_new.
- Attach the same volume projectdata at /app/data.
- Verify that report.txt still exists and contains the same data.



- `docker volume create projectdata`
- `docker volume ls`
- `docker volume inspect projectdata`
- `docker run -dit --name project_container -v projectdata:/app/data ubuntu`
- `docker exec -it project_container sh -c "echo This is my first volume" > /app/data/report.txt`
- `docker exec -it project_container cat /app/data/report.txt`
- `docker stop project_container`
- `docker rm project_container`
- `docker run -dit --name project_container_new -v projectdata:/app/data ubuntu`
- `docker exec -it project_container_new ls /app/data`
- `docker exec -it project_container_new cat /app/data/report.txt`

Task 1



You are a DevOps engineer at a university. The web development team has built a simple HTML-based portal and wants it deployed using Docker.

- Use the official httpd image.
- Name the container college_portal.
- Run it in detached mode.
- Expose container port 80 to host port 8080.
- Set environment variable ENV=production.
- Attach a Docker volume named portaldata to store website files.
- Modify the index.html file inside the container.
- Verify the output in the browser.
- Check logs if the page is not loading.
- Stop and remove the container.
- Remove the volume and image after testing.

- `docker pull httpd`
- `docker volume create portaldata`
- `docker run -dit --name college_portal -p 8080:80 -e ENV=production -v portaldata:/usr/local/apache2/htdocs httpd`
- `docker exec -it college_portal bash -c "echo '<h1>Welcome to University Portal - Production Mode</h1>' >/usr/local/apache2/htdocs/index.html"`
- `curl http://localhost:8080`
- `docker logs college_portal`
- `docker logs -f college_portal`
- `docker stop college_portal`
- `docker rm college_portal`
- `docker volume rm portaldata`
- `docker rmi httpd`

Task 2



- Your backend team reports that their MySQL container is failing to start.
- Use mysql:8 image.
- Set required environment variables:
 - MYSQL_ROOT_PASSWORD
 - MYSQL_DATABASE
 - MYSQL_USER
 - MYSQL_PASSWORD
- Run in background.
- Expose port 3306 to host port 3307.
- Check logs to identify errors.
- Enter inside container to verify database.
- Stop and remove container after debugging.



- `docker pull mysql:8`
- `docker run -d --name mysql_debug -p 3307:3306 -e MYSQL_ROOT_PASSWORD=root123 -e MYSQL_DATABASE=college -e MYSQL_USER=admin -e MYSQL_PASSWORD=admin123 mysql:8`
- `docker ps`
- `docker logs mysql_debug`
- `docker logs -f mysql_debug`
- `docker exec -it mysql_debug bash`
- `mysql -u root -p // # (Enter password: root123)`
- `exit`
- `Exit`
- `docker stop mysql_debug`
- `docker rm mysql_debug`



Image creation in detail



- `mkdir nginx-app`
- `cd nginx-app`
- `notepad index.html`

- `<!DOCTYPE html>`
- `<html>`
- `<head>`
- `<title>Medium Level Docker App</title>`
- `</head>`
- `<body>`
- `<h1>Welcome to My Custom Docker Nginx App</h1>`
- `<p>This is a medium-level Docker project.</p>`
- `</body>`
- `</html>`

- notepad default.conf

```
server {  
    listen 80;  
    server_name localhost;  
  
    location / {  
        root /usr/share/nginx/html;  
        index index.html;  
    }  
}
```

- notepad Dockerfile

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

COPY default.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

- Dir
- `docker build -t custom-nginx:v1 .`
- `docker run -d -p 8080:80 --name nginx-container custom-nginx:v1`
- `http://localhost:8080`



- `mkdir node-app`
- `cd node-app`

- `apt update`
- `apt install nano -y`

- Create app.js

```
const express = require("express");  
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("Docke Node App Running!");  
});
```

```
app.listen(3000, "0.0.0.0", () => {  
  console.log("Server running on port 3000");  
});
```

- Create package.json

```
{  
  "name": "node-docker-app",  
  "version": "1.0.0",  
  "main": "app.js",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

- Create Dockerfile

FROM node:18-alpine

WORKDIR /app

COPY package.json .

RUN npm install

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]

Verify files



- Dir

app.js

package.json

Dockerfile- **Rename-Item Dockerfile.txt Dockerfile**

Build & run & verify



- `docker build -t node-demo:v1 .`
- `Docker run -d -p 3000:3000 --name node-container node-demo:v1`
- `docker ps`
- OPEN BROWSER

<http://localhost:3000>

Docker Node App Running!



- Docker provides different network drivers to control how containers communicate.

A. Bridge Network (Default)

- Default network when you run a container.
- Containers on same bridge can communicate via IP.
- Best for standalone containers on single host.



List networks

```
docker network ls
```

Inspect bridge network

```
docker network inspect bridge
```

Create custom bridge network

```
docker network create my_bridge_net
```

Run container in custom bridge

```
docker run -d --name container1 --network my_bridge_net nginx
```

```
docker run -d --name container2 --network my_bridge_net alpine sleep 300
```

Test communication

```
docker exec -it container2 ping container1
```

- Container shares host's networking stack.
- No separate IP.
- Faster performance.
- Linux only.

```
docker run -d --network host nginx  
curl http://localhost
```

- Used in Docker Swarm.
- Connects containers across multiple hosts.

Initialize swarm

```
docker swarm init
```

Create overlay network

```
docker network create -d overlay my_overlay
```

Deploy service using overlay

```
docker service create --name web --network my_overlay -p 8080:80 nginx
```



- Docker has built-in DNS.
- Containers resolve each other by container name (if same custom network).
- Works only in user-defined bridge networks.

- `docker network create app_net`
- `docker run -d --name backend --network app_net nginx`
- `docker run -it --name client --network app_net alpine sh`
- `ping backend`



- Before custom networks, Docker used `--link`.
- Deprecated method. Use networks instead.

```
docker run -d --name db nginx
```

```
docker run -it --name app --link db:database alpine sh
```

Create volume

```
docker volume create my_volume
```

List volumes

```
docker volume ls
```

Run container with volume

```
docker run -d \
```

```
--name my_nginx \
```

```
-v my_volume:/usr/share/nginx/html \
```

```
nginx
```



- `docker volume inspect my_volume`
- `docker volume rm my_volume`

- Mounts host directory into container
- Useful for development

```
docker run -d \  
-v $(pwd)/html:/usr/share/nginx/html \  
-p 8080:80 \  
nginx
```

Volume Vs Bind Mount



Complete Example



- `docker network create app_bridge`
- `docker network ls`
- `docker volume create mysql_data`
- `docker volume inspect mysql_data`
- `docker run -d --name mysql_db --network app_bridge -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=myapp -v mysql_data:/var/lib/mysql mysql:8`
- `docker run -d --name backend_app --network app_bridge -e DB_HOST=mysql_db node:18-alpine sh -c "apk add curl && sleep 300"`



- `docker exec -it backend_app ping mysql_db`
- `docker run -d --name frontend --network app_bridge -p 8080:80 nginx`